Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División Ciencias de la Ingeniería
Base de Datos 2



“Manual Técnico”

Kenny Marco Augusto López Salazar **202031554**

Quetzaltenango 30 de octubre 2026

MANUAL TÉCNICO

En este documento describo la arquitectura, las tecnologías, la estructura de datos, el proceso de carga y las verificaciones del sistema de análisis de pruebas específicas de ingreso. El objetivo es facilitar su instalación, mantenimiento y explicación durante la evaluación.

El proyecto procesa archivos CSV de 2015 a 2024 y presenta consultas y gráficos a través de una aplicación web. Las capturas registran etapas de desarrollo y pruebas; los valores de una carga parcial no deben confundirse con los resultados finales.

1. Arquitectura y tecnologías

Organicé la aplicación en tres componentes. React presenta los resultados y envía peticiones HTTP al backend. La API implementada con Java 21 y Spring Boot valida los parámetros y consulta MongoDB o Neo4j. Las bases de datos se ejecutan en contenedores Docker con volúmenes para conservar sus datos.

Utilicé MongoDB 7.0 para almacenar las evaluaciones como documentos y realizar agregaciones. Utilicé Neo4j 5.26 Community para representar aspirantes, evaluaciones y relaciones. El frontend no se conecta directamente a las bases; todas las operaciones pasan por la API.

También utilicé Maven Wrapper para ejecutar y construir el backend, Node.js y npm para el frontend, Docker Compose para iniciar las bases, Git y GitHub para mantener el historial de cambios, y Python 3 para scripts de verificación y auditoría. Las pruebas se realizaron en Ubuntu 24.04 con Java 21 y Node.js 24.

MongoDB 8.0 presentó una incompatibilidad con el kernel del equipo durante el arranque; por eso se utilizó MongoDB 7.0, cuya ejecución se verificó en este entorno. La elección de versión es una decisión de compatibilidad del equipo de desarrollo.

2. Infraestructura y puertos

El archivo de Docker Compose define los servicios y sus volúmenes. MongoDB se expone en 127.0.0.1:27018 y Neo4j en 127.0.0.1:7474 para la interfaz HTTP y 127.0.0.1:7687 para Bolt. El backend atiende en el puerto 8091 y el frontend de desarrollo en 5173.

Los puertos de las bases se enlazan a la interfaz local para el entorno de desarrollo. Las credenciales y el nombre de la base se configuran mediante el archivo .env local, que no debe incluirse en el repositorio. Un volumen de Docker conserva los datos al detener e iniciar los contenedores.

```
cd ~/Documentos/ProyectoFinal_Bases2
docker compose config --quiet
docker compose up -d
docker compose ps
```

El estado healthy indica que los contenedores superaron su comprobación de salud. Durante el arranque puede mostrarse health: starting; espero a que termine antes de consultar los servicios.

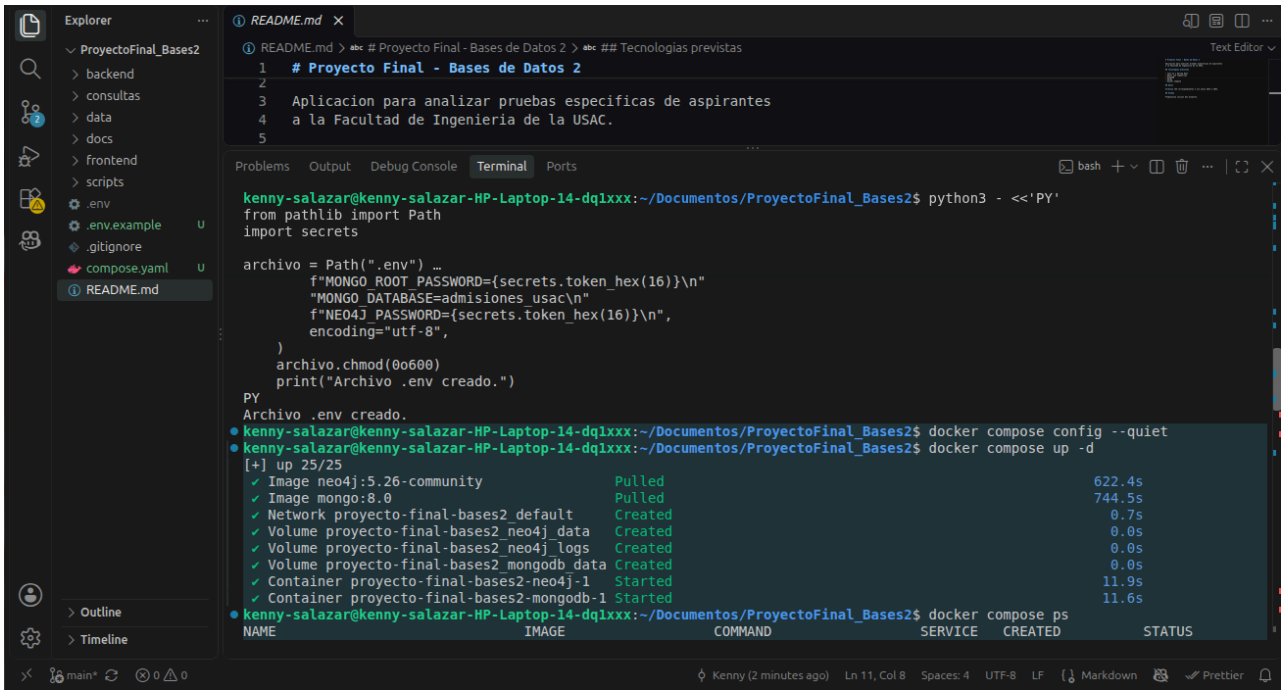


Figura 1. Inicio y comprobación de los contenedores

En esta etapa revisé la configuración de Compose e inicié los servicios. La captura corresponde a la preparación inicial de la infraestructura; el resultado definitivo se confirmó después con las comprobaciones de conexión.

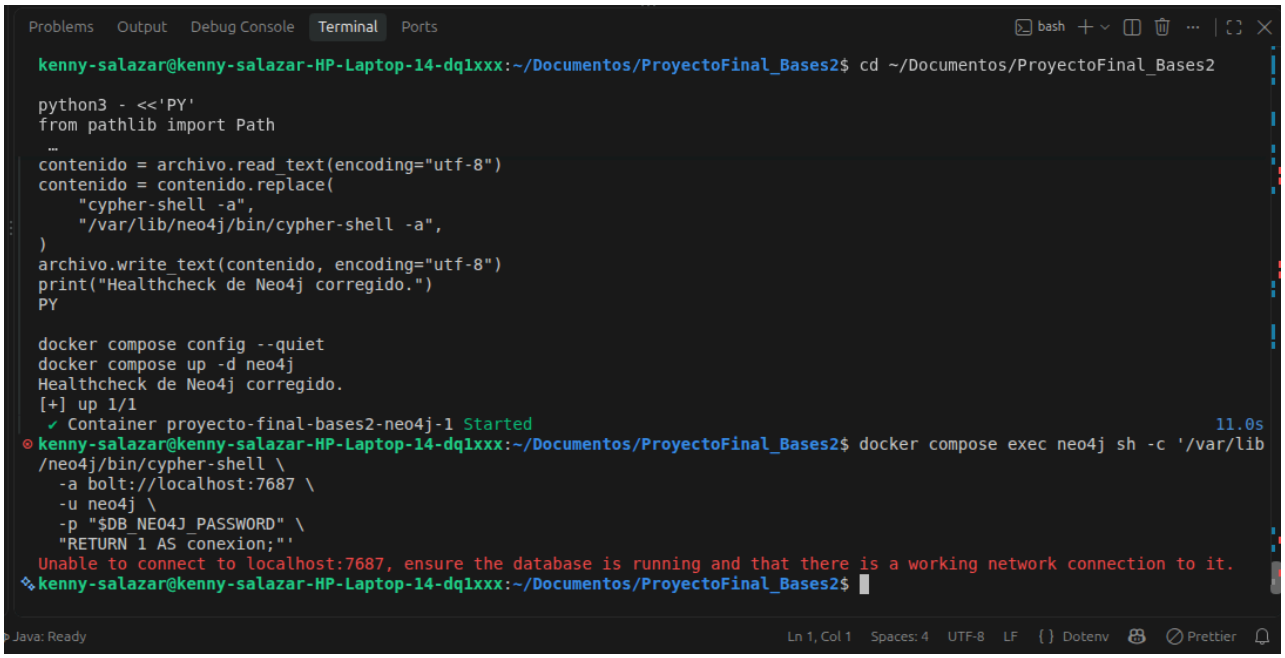


Figura 2. Diagnóstico inicial de Neo4j

Esta captura muestra un error de conexión durante el arranque. Lo registré como parte del diagnóstico y posteriormente comprobé la conexión con cypher-shell y el estado healthy; la imagen no representa el estado final del servicio.

3. Puesta en marcha

Después de configurar el archivo .env local, inicio las bases con Docker Compose. En una terminal separada ejecuto el backend desde su carpeta; el script carga las variables necesarias y utiliza Maven Wrapper.

cd ~/Documentos/ProyectoFinal_Bases2/backend
./ejecutar.sh

En otra terminal instalo las dependencias del frontend y ejecuto Vite. npm ci se utiliza al preparar el proyecto o después de actualizar dependencias. Mantengo abiertas ambas terminales durante el uso.

cd ~/Documentos/ProyectoFinal_Bases2/frontend
npm ci
npm run dev

Para revisar la API consulto el endpoint de estado. Cuando ambos servicios responden devuelve HTTP 200 con estado DISPONIBLE. Si una base no responde, puede devolver HTTP 503 y estado DEGRADADO.

curl -i http://localhost:8091/api/v1/estado

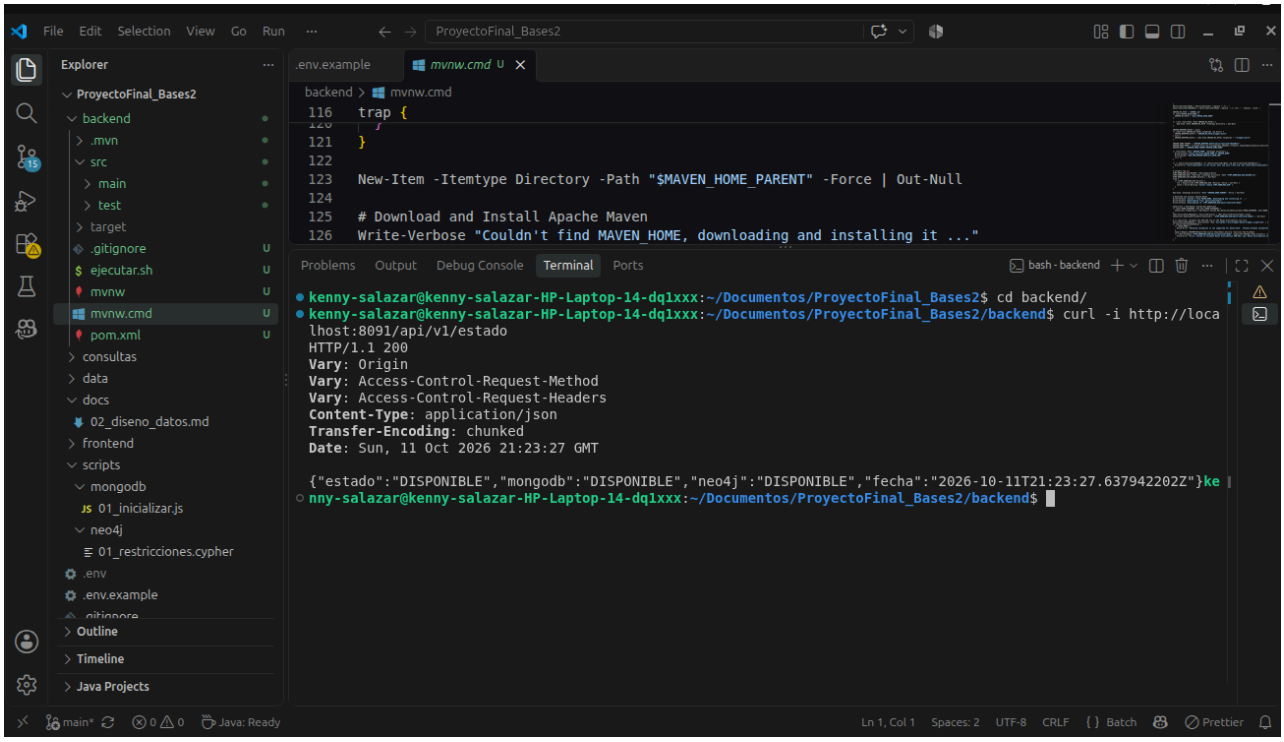


Figura 3. Comprobación del endpoint de estado

La respuesta de la API me permitió comprobar que MongoDB y Neo4j estaban disponibles desde el backend. La comprobación evalúa conectividad; por sí sola no verifica que todos los archivos estén cargados.

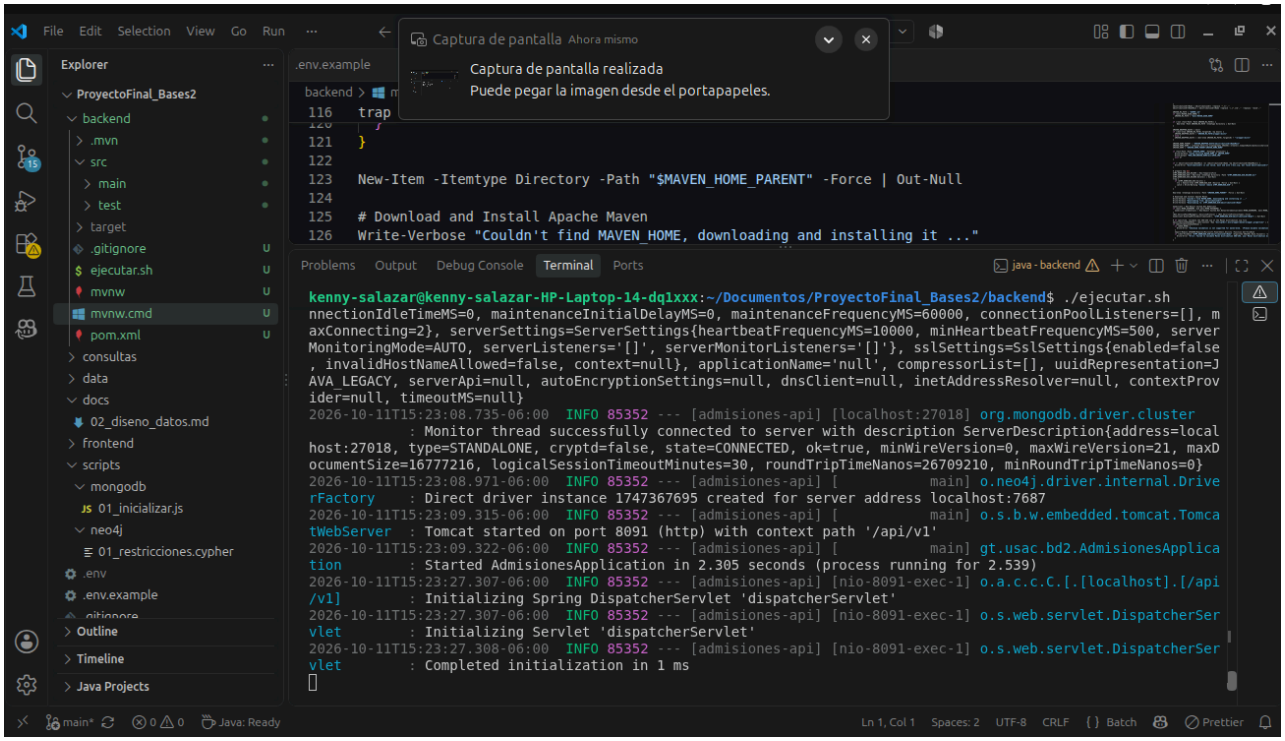


Figura 4. Arranque del backend

La terminal registra el inicio de Spring Boot y la actividad de los controladores y conexiones. Mantener esta terminal activa permite atender las solicitudes del frontend.

4. Organización del proyecto

La carpeta backend contiene la aplicación Spring Boot, su configuración y pruebas; frontend contiene la interfaz React. En scripts se encuentran inicialización y verificaciones; consultas contiene archivos de consulta legibles para los motores; docs reúne el diseño y las guías. Docker Compose y .env se ubican en la raíz del repositorio.

En el backend, el catálogo principal de las 20 consultas MongoDB está en backend/src/main/resources/consultas/mongodb.json. Las copias independientes se generan en consultas/mongodb mediante scripts/generar_consultas_mongodb.py. Las cinco consultas Neo4j tienen su catálogo en backend/src/main/resources/consultas/neo4j.json y copias en consultas/neo4j. La API utiliza los catálogos del backend.

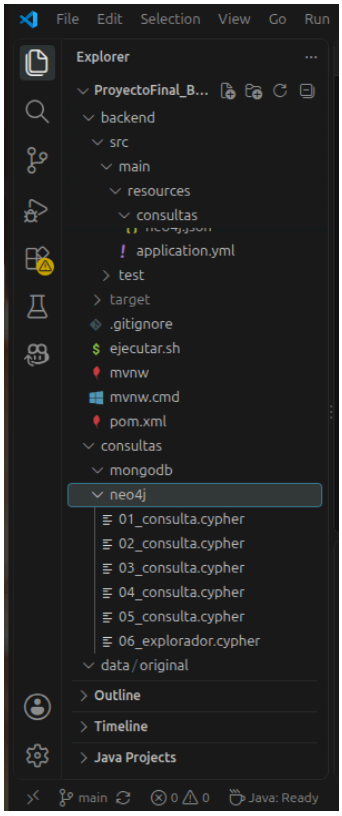


Figura 5. Organización de las consultas

En el explorador de VS Code se observa la carpeta de consultas y los archivos de Neo4j. Esta organización me permite revisar las sentencias junto con el código y conservar copias para su ejecución o documentación.

5. Diseño de MongoDB

La base admisiones_usac contiene evaluaciones, cargas, errores_carga y resumen_carrera. Cada evaluación conserva los campos normalizados para consultar, los valores originales, advertencias, identificador de carga y estado de sincronización con Neo4j. Las cargas guardan contadores y estado; los errores permiten revisar filas rechazadas.

Definí un índice único para evaluacion_id y otros índices para historial de aspirante, año y carrera, sincronización, cargas y errores. El identificador de evaluación se calcula mediante SHA-256 de los valores originales en el orden canónico; una fila idéntica mantiene el mismo identificador aunque se vuelva a cargar el archivo.

resumen_carrera es una colección auxiliar materializada. Se debe regenerar después de nuevas cargas para que la consulta 6 muestre los datos actualizados. Las veinte consultas de MongoDB incluyen agrupaciones, porcentajes, historial y análisis por procedencia; se paginan los resultados.

6. Diseño de Neo4j

El grafo utiliza nodos Aspirante, Evaluacion, Materia, Carrera, TipoInstitucion, Municipio y Departamento. Las relaciones conectan al aspirante con su evaluación, y a la evaluación con materia, carrera y procedencia institucional. El municipio se relaciona con su departamento.

Las restricciones de unicidad protegen las claves de los siete tipos de nodo. MERGE permite reintentar la sincronización sin crear nuevamente los mismos nodos o relaciones. En MongoDB se confirma el estado SINCRONIZADO después de completar el lote en Neo4j.

Implementé cinco consultas para trayectoria del aspirante, rendimiento por materia y carrera, procedencia y carrera, materias compartidas y recuperación en oportunidades posteriores. El grafo visual muestra una muestra limitada; sus consultas analíticas trabajan sobre los registros que cumplen los filtros.

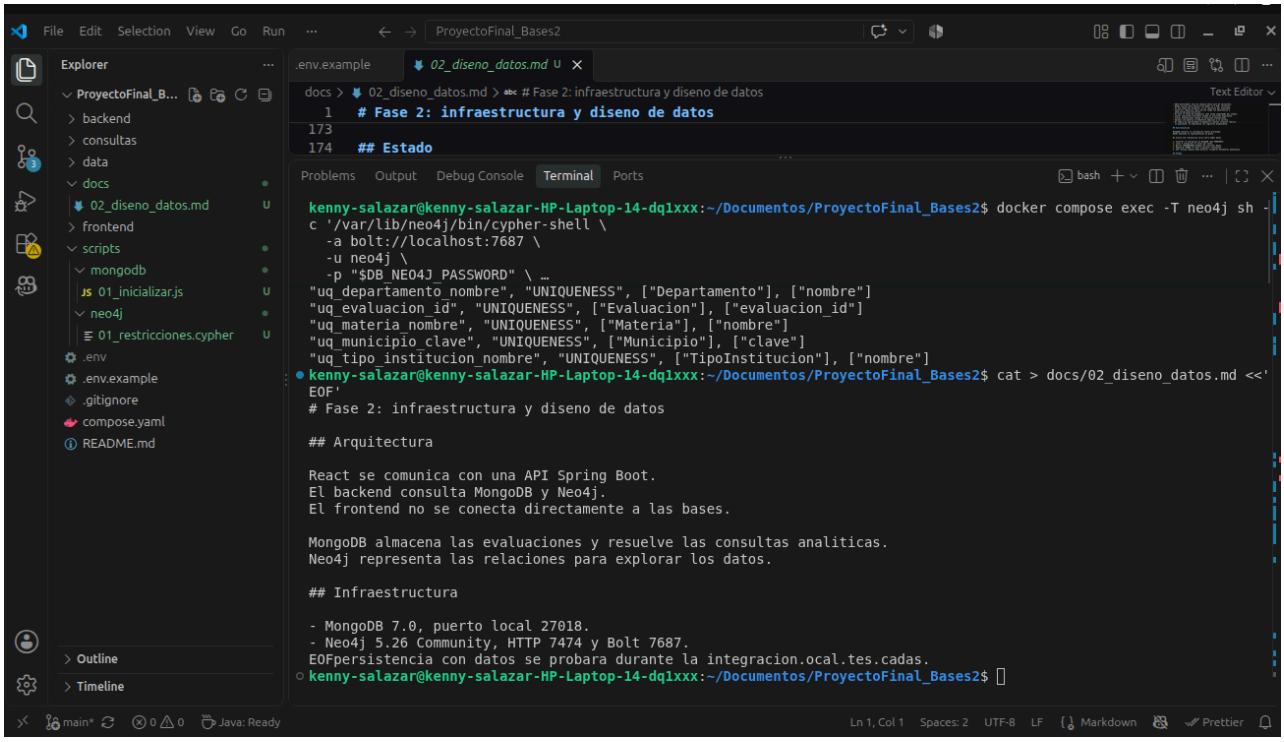


Figura 6. Restricciones y diseño inicial

La captura documenta la ejecución de restricciones y el registro del diseño de datos. Estas restricciones ayudan a impedir duplicaciones de claves en el grafo.

7. Carga y validación de archivos CSV

La carga acepta archivos CSV UTF-8 y reconoce los doce campos previstos, incluidas variantes de encabezados de años recientes. El backend valida campos obligatorios, fecha, año, oportunidad, sexo y resultado. Conserva datos originales y normalizados para facilitar la revisión.

Las filas válidas nuevas se insertan en MongoDB; las filas exactas que ya existen se cuentan como duplicadas y las inválidas se registran en errores_carga. Una advertencia describe una condición de

calidad en un registro válido, por ejemplo un año de nacimiento sin registro o un posible texto truncado. La advertencia no equivale a rechazo.

Al terminar la importación, el sistema intenta sincronizar los registros pendientes hacia Neo4j por lotes. Si Neo4j no está disponible, las evaluaciones confirmadas permanecen en MongoDB y se puede utilizar Reintentar Neo4j cuando el servicio se recupere. El estado histórico de un intento no necesariamente cambia cuando se completa un reintento posterior.

En el conjunto original identifiqué 208453 filas, de las cuales 207619 son únicas y 834 son duplicados exactos. En las evaluaciones únicas se verificaron 29701 aprobadas y 177918 no aprobadas. Una aprobación corresponde a una prueba, no a la admisión completa del aspirante.

8. Validación funcional y consistencia

Los scripts de Python facilitan la repetición de pruebas sin instalar controladores adicionales. verificar_fase5.py revisa las veinte consultas MongoDB y puede regenerar el resumen; verificar_fase6.py comprueba las cinco consultas y el grafo; verificar_fase7.py prueba buscador, filtros, historial y detalle.

```
export ASPIRANTE='949d241ac10256edf11339a303c72a50'
python3 scripts/verificar_fase5.py --aspirante "$ASPIRANTE" --generar-resumen
python3 scripts/verificar_fase6.py --aspirante "$ASPIRANTE"
python3 scripts/verificar_fase7.py --aspirante "$ASPIRANTE"
```

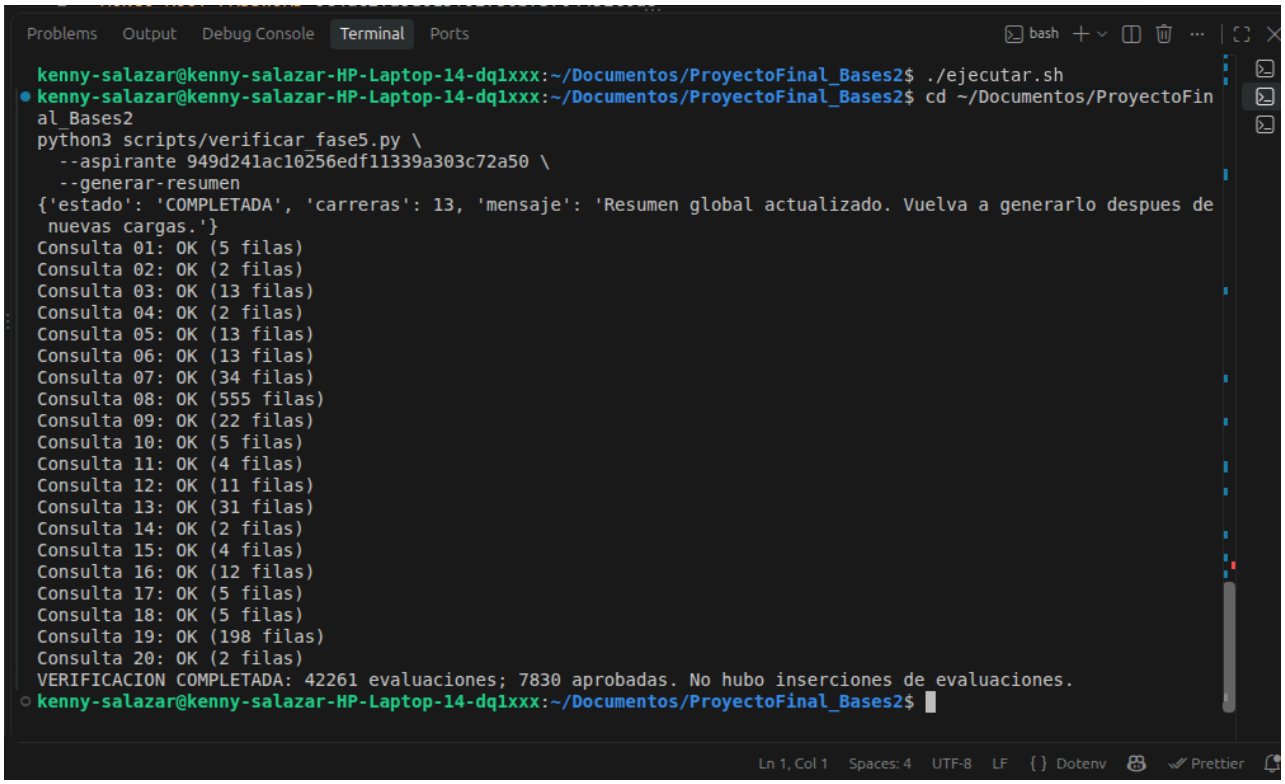


Figura 7. Verificación de consultas MongoDB

La captura muestra una prueba inicial con datos parciales. Después ejecuté nuevamente la verificación con el dataset completo y las veinte consultas terminaron correctamente.

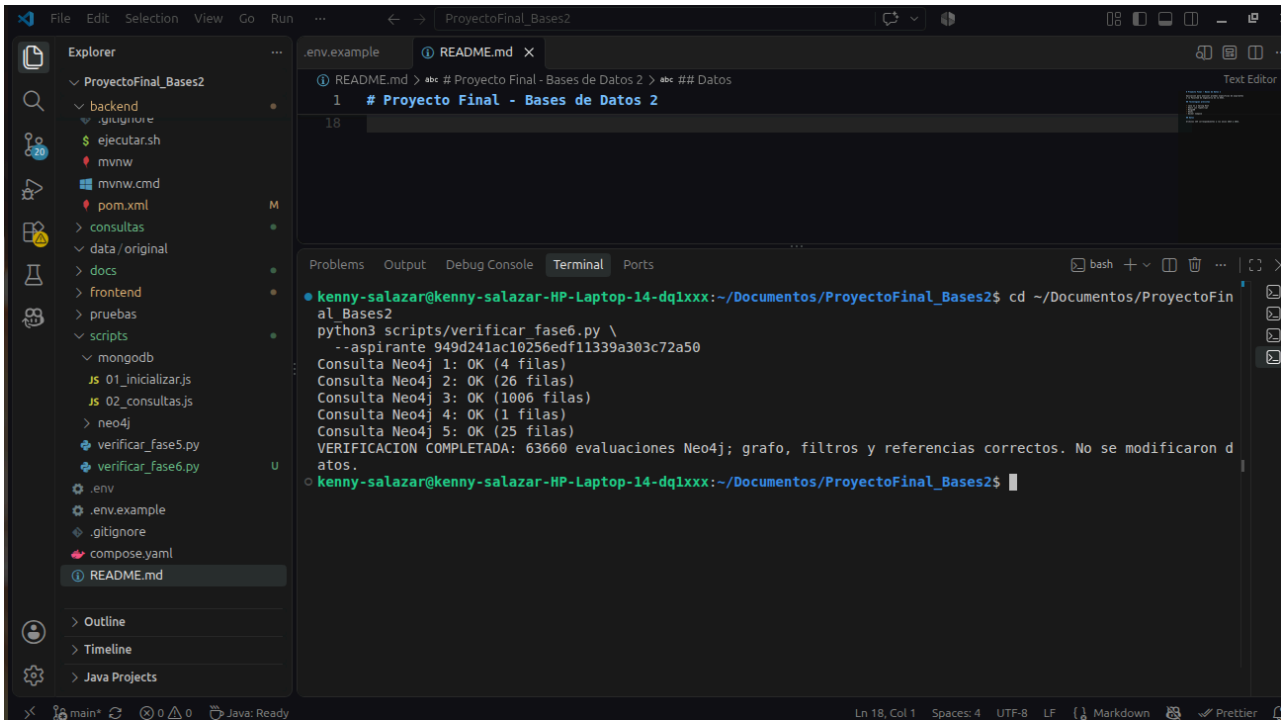


Figura 8. Verificación de consultas Neo4j

La terminal muestra la ejecución de las cinco consultas y las comprobaciones del grafo. La captura corresponde a una carga parcial; la prueba también se repitió con el conjunto completo.

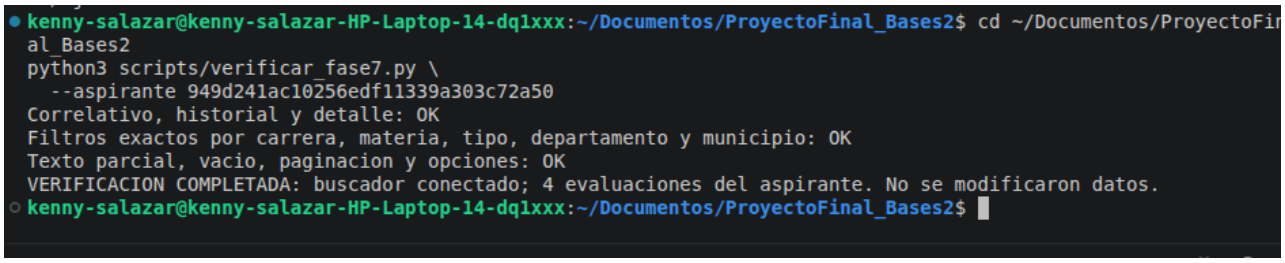


Figura 9. Verificación del buscador

Comprobé correlativo, historial, detalle, filtros exactos, búsqueda parcial y paginación. La prueba de lectura identificó cuatro evaluaciones para el correlativo de ejemplo.

9. Auditoría, persistencia y rendimiento

auditar_dataset.py compara los identificadores, años y resultados de los CSV con MongoDB y Neo4j. También comprueba que no haya sincronizaciones pendientes y revisa la cardinalidad de las relaciones principales. No compara todos los atributos ni todos los nodos auxiliares aislados.

python3 scripts/auditar_dataset.py --csv ~/Descargas/ArchivosCSV.zip \
+ --salida reportes/fase8/antes_reinicio.json

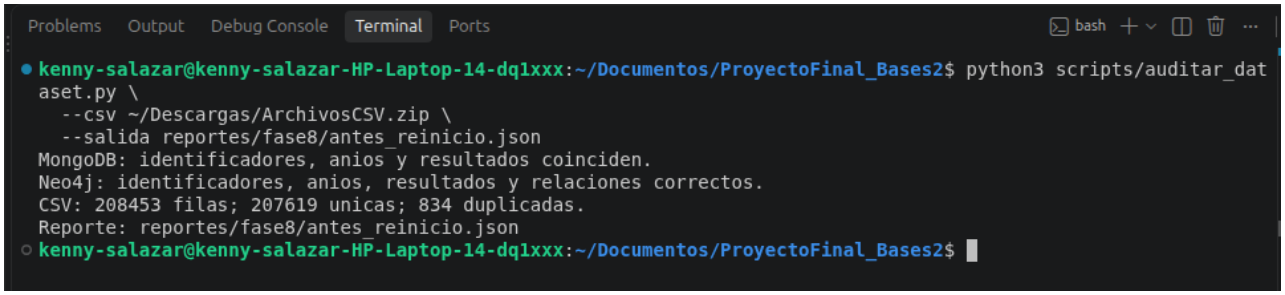


Figura 10. Auditoría de los datos cargados

La auditoría confirmó que los identificadores, años y resultados coincidían en ambas bases y que las relaciones verificadas eran correctas para las 207619 evaluaciones únicas.

Probé la persistencia deteniendo e iniciando los dos contenedores, sin eliminar sus volúmenes. Después volví a ejecutar la auditoría y comparé el resultado con el reporte anterior. Esta prueba verifica el reinicio de los contenedores; un reinicio completo del sistema operativo es un escenario adicional.

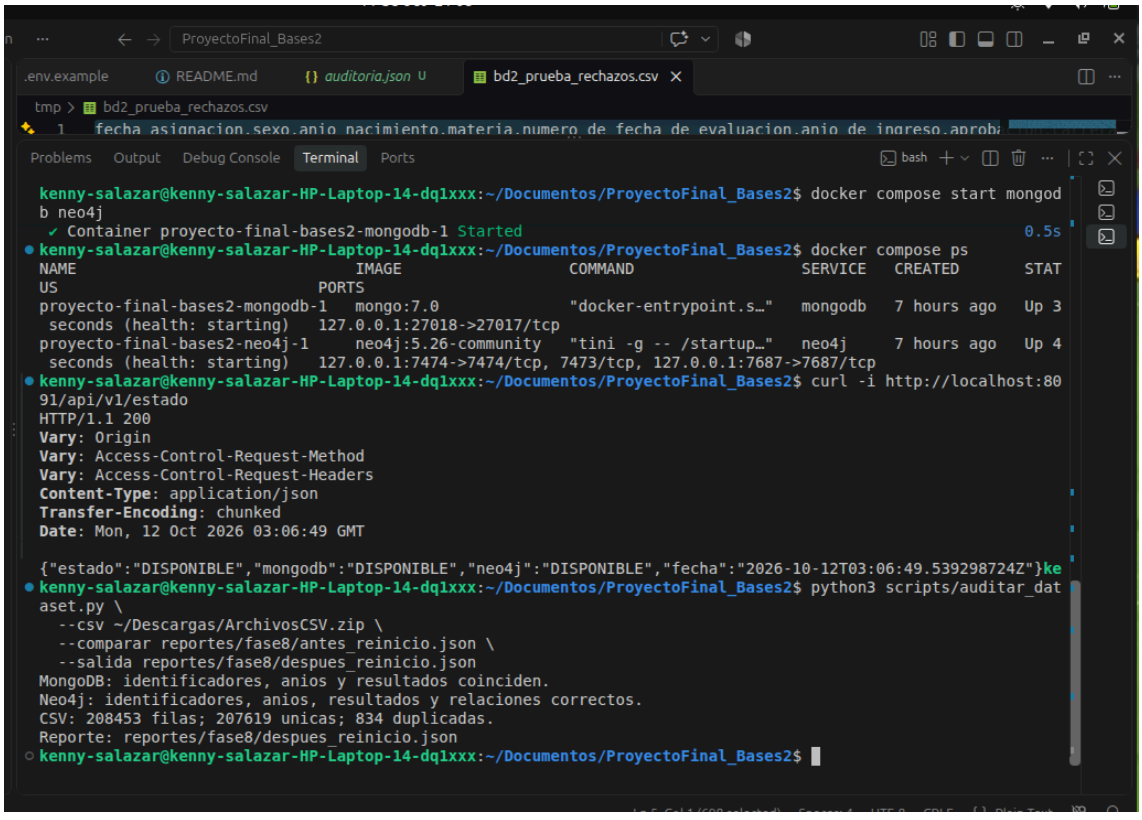


Figura 11. Persistencia después del reinicio

La captura muestra el reinicio de los servicios, la respuesta del endpoint de estado y la auditoría posterior. Los identificadores, años, resultados y relaciones revisadas siguieron coincidiendo.

medir_consultas.py ejecutó dos solicitudes por cada una de las 20 consultas MongoDB y cinco Neo4j. Las medianas observadas fueron de 4.37 a 1168.39 ms en MongoDB y de 2.56 a 7.43 segundos en

Neo4j. Se midió el tiempo HTTP de una página e incluye API, agregación, conteo y serialización; no es una prueba de carga concurrente.

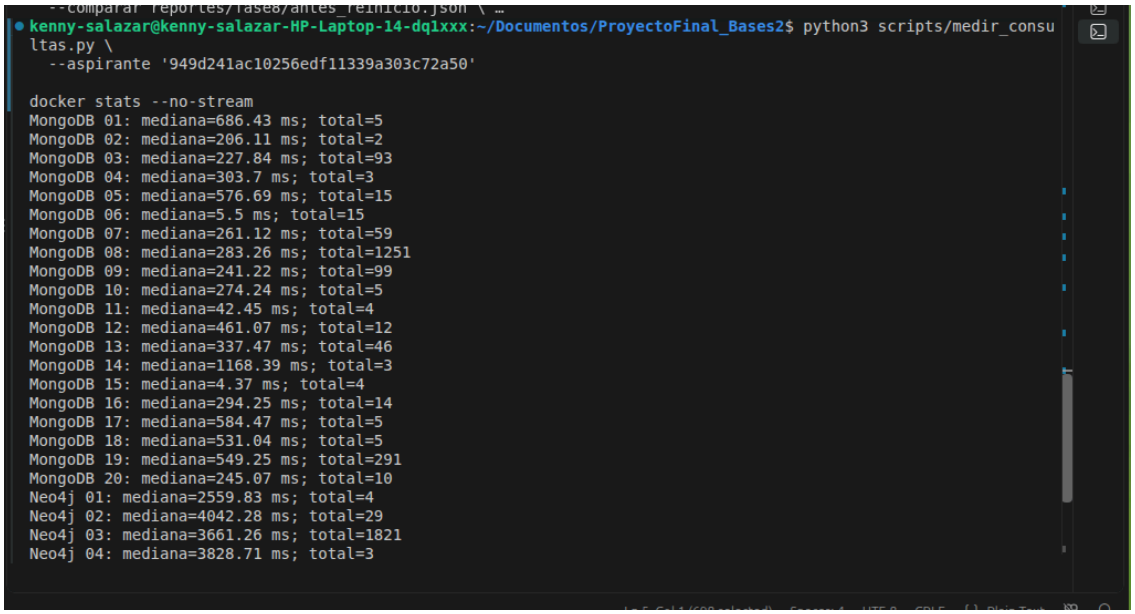


Figura 12. Medición de consultas

El reporte registró las consultas y sus tiempos. Las cifras sirven como referencia de la laptop y la carga de datos probadas, no como garantía de rendimiento en otros equipos.

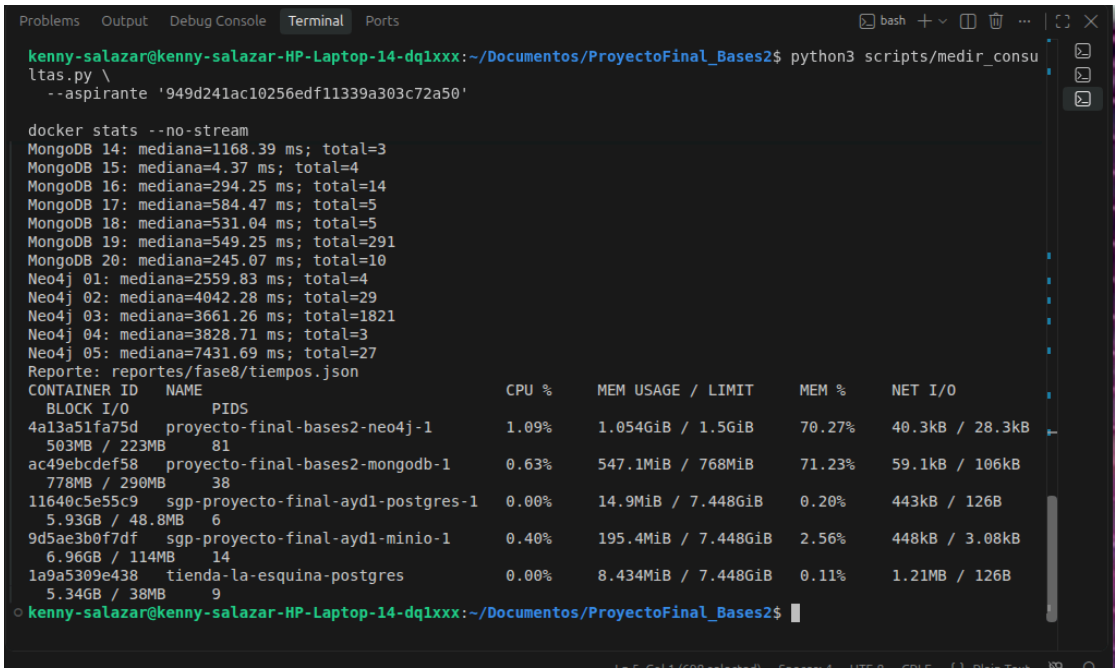


Figura 13. Consumo de recursos

docker stats mostró una instantánea: Neo4j utilizó 1.054 GiB de un límite de 1.5 GiB y MongoDB 547.1 MiB de 768 MiB. Es una lectura puntual posterior a las mediciones, no el máximo de toda la ejecución.

10. Mantenimiento y solución de problemas

Si una base no inicia, reviso docker compose ps y docker compose logs --tail=60 mongodb neo4j. Si la API responde degradada, verifico los servicios, las variables locales y la conectividad. Si hay registros pendientes, recupero Neo4j y reintento la sincronización desde la aplicación.

Después de cargar datos nuevos, regenero resumen_carrera y repito las verificaciones. Para consultar el detalle de rechazos reviso el historial de cargas y los registros de errores. El archivo de prueba puede contener filas inválidas deliberadamente y debe mantenerse separado de los originales.

Para detener los servicios uso docker compose stop mongodb neo4j. Evito eliminar volúmenes como parte del cierre habitual. Un volumen persistente no sustituye una copia de seguridad: para proteger los datos ante pérdida del equipo se necesita un procedimiento de respaldo y restauración probado.

11. Control de versiones y conclusiones

Utilicé Git para registrar los cambios por fases y GitHub como repositorio remoto. Antes de confirmar cambios conviene revisar git status y el contenido agregado, sin incluir el archivo .env, credenciales ni datos que no deban publicarse.

La solución combina documentos para consultas agregadas y un grafo para explorar relaciones. Las validaciones realizadas comprobaron la carga completa, el reconocimiento de duplicados, el rechazo de filas inválidas, la recuperación de sincronización y la persistencia tras reiniciar contenedores. Los tiempos de consulta de Neo4j quedaron registrados como una referencia para futuras mejoras.